Self-Driving Cars

Exercise 3 - Reinforcement Learning

Bernhard Jaeger
Autonomous Vision Group
University of Tübingen

EBERHARD KARLS
UNIVERSITÄT
TÜBINGEN







ellis
European Laboratory for Learning and Intelligent Systems

Exercise Setup

Download `exercise_03_reinforcement_learning.zip` which contains:

- Exercise sheet & slides
- Code template

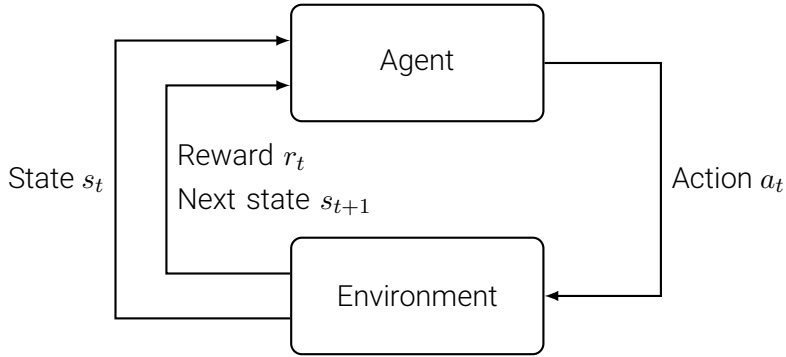
Submit `.zip` folder which contains:

- Your best model (`agent.pth`)
- Your Python codes (`.py`)
- Your action file (`default_actions.txt`)
- Put all files in the root folder

Deadline: **Tuesday, 10. December 2025 - 21:00**

Reinforcement Learning

Reinforcement Learning



- Agent oberserves environment state s_t at time t
- Agent performs action a_t at time t
- Environment returns the reward r_t and its new state s_{t+1} to the agent

Machine Learning, 8, 279–292 (1992)
© 1992 Kluwer Academic Publishers, Boston. Manufactured in The Netherlands.

Technical Note

Q-Learning

CHRISTOPHER J.C.H. WATKINS
25b Framfield Road, Highbury, London N5 1UU, England

PETER DAYAN
Centre for Cognitive Science, University of Edinburgh, 2 Buccleuch Place, Edinburgh EH8 9EH, Scotland

Abstract. Q-learning (Watkins, 1989) is a simple way for agents to learn how to act optimally in controlled Markovian domains. It amounts to an incremental method for dynamic programming which imposes limited computational demands. It works by successively improving its evaluations of the quality of particular actions at particular states.

This paper presents and proves in detail a convergence theorem for Q-learning based on that outlined in Watkins (1989). We show that Q-learning converges to the optimum action-values with probability 1 so long as all actions are repeatedly sampled in all states and the action-values are represented discretely. We also sketch extensions to the cases of non-discounted, but absorbing, Markov environments, and where many Q values can be changed each iteration, rather than just one.

Keywords. Q-learning, reinforcement learning, temporal differences, asynchronous dynamic programming

1. Introduction

Q-learning (Watkins, 1989) is a form of model-free reinforcement learning. It can also be viewed as a method of asynchronous dynamic programming (DP). It provides agents with



LETTER

doi:10.1038/nature14236

Human-level control through deep reinforcement learning

Volodymyr Mnih[1]*, Koray Kavukcuoglu[1]*, David Silver[1]*, Andrei A. Rusu[1], Joel Veness[1], Marc G. Bellemare[1], Alex Graves[1], Martin Riedmiller[1], Andreas K. Fidjeland[1], Georg Ostrovski[1], Stig Petersen[1], Charles Beattie[1], Amir Sadik[1], Ioannis Antonoglou[1], Helen King[1], Dharshan Kumaran[1], Daan Wierstra[1], Shane Legg[1] & Demis Hassabis[1]

The theory of reinforcement learning provides a normative account[1], deeply rooted in psychological[2] and neuroscientific[3] perspectives on animal behaviour, of how agents may optimize their control of an environment. To use reinforcement learning successfully in situations approaching real-world complexity, however, agents are confronted with a difficult task: they must derive efficient representations of the environment from high-dimensional sensory inputs, and use these to generalize past experience to new situations. Remarkably, humans and other animals seem to solve this problem through a harmonious combination of reinforcement learning and hierarchical sensory processing systems[4,5], the former evidenced by a wealth of neural data revealing notable parallels between the phasic signals emitted by dopaminergic neurons and temporal difference reinforcement learning algorithms[3]. While reinforcement learning agents have achieved some successes in a variety of domains[6–8], their applicability has previously been limited to domains in which useful features can be handcrafted, or to domains with fully observed, low-dimensional state spaces. Here we use recent advances in training deep neural networks[9–11] to develop a novel artificial agent, termed a deep Q-network, that can learn successful policies directly from high-dimensional sensory inputs using end-to-end reinforcement learning. We tested this agent on the challenging domain of classic Atari 2600 games[12]. We demonstrate that the deep Q-network agent, receiving only the pixels and the game score as inputs, was able to surpass the performance of all previous algorithms and achieve a level comparable to that of a professional human games tester across a set of 49 games, using the same algorithm, network architecture and hyperparameters. This work bridges the divide between high-dimensional sensory inputs and actions, resulting in the first artificially intelligent agent that is capable of learning to excel at a diverse array of challenging tasks.

agent is to select actions in a fashion that maximizes cumulative future reward. More formally, we use a deep convolutional neural network to approximate the optimal action-value function

$$Q^*(s,a) = \max_{\pi} \mathbb{E}\left[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \ldots \mid s_t = s,\, a_t = a,\, \pi\right],$$

which is the maximum sum of rewards r_t discounted by γ at each time-step t, achievable by a behaviour policy $\pi = P(a|s)$, after making an observation (s) and taking an action (a) (see Methods)[19].

Reinforcement learning is known to be unstable or even to diverge when a nonlinear function approximator such as a neural network is used to represent the action-value (also known as Q) function[20]. This instability has several causes: the correlations present in the sequence of observations, the fact that small updates to Q may significantly change the policy and therefore change the data distribution, and the correlations between the action-values (Q) and the target values $r + \gamma \max_{a'} Q(s', a')$. We address these instabilities with a novel variant of Q-learning, which uses two key ideas. First, we used a biologically inspired mechanism termed experience replay[21–23] that randomizes over the data, thereby removing correlations in the observation sequence and smoothing over changes in the data distribution (see below for details). Second, we used an iterative update that adjusts the action-values (Q) towards target

Deep Q-network

Use a deep neural network with weights θ to estimate $Q(s, a; \theta) \approx Q^*(s, a)$:



Deep Q-Learning

Training a deep Q-network using **experience replay** and **fixed Q-targets**

- Take action a_t according to ϵ-greedy policy
- Store transition (s_t, a_t, r_t, s_{t+1}) in replay buffer D
- Sample random mini-batch of transitions (s, a, r, s') from D
- Compute Q-learning targets w.r.t. old, fixed parameters θ^-
- Optimize MSE between Q-network and Q-learning targets:

$$L_i(\theta_i) = \mathbb{E}_{s,a,r,s' \sim D_i} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta_i^-) - Q(s, a; \theta_i) \right)^2 \right]$$

using a variant of stochastic gradient descent

1.1
Base Implementation

Code Template

The provided code template contains:

- Deep Q-network (**to-do**)
- Deep Q-learning (**to-do**)
- Action selection (**to-do**)
- Training (implemented, incl. replay buffer, exploration)
- Evaluation (implemented)

a) Deep Q-network

Implement a deep Q-network and its forward pass:

- Start with a simple network architecture
  - Some convolution + fully connected layers
  - Probably no need for batch normalization, dropout or residual architectures
  - Use a single frame as input to the network
  - You may again use the `extract_sensor_values` function
- Get inspired by the original DQN architecture for playing Atari
- Try to adapt your network architecture from Exercise 1

b) Deep Q-learning

Implement the deep Q-learning update step *(see DQN Nature paper for details)*:

1. Sample transitions from replay buffer
2. Compute $Q(s_t, a)$
3. Compute $\max_a Q(s_{t+1}, a)$ for all next states
4. Mask next state values where episodes have terminated
5. Compute the target and loss
6. Calculate and clip the gradients
7. Optimize the model

Implement the target update

c) Action selection

Implement selecting an exploratory ϵ-greedy or greedy action:

- With probability ϵ choose an action at random
- With probability $1 - \epsilon$ choose the greedy action:

$$\pi(a|s) = \begin{cases} 1 & \text{if } a = \underset{a \in \mathcal{A}}{\operatorname{argmax}}\ Q(s, a; \theta) \\ 0 & \text{otherwise} \end{cases}$$

d) Training

Train a deep Q-learning agent:

- Use script `train_racing.py` to train
- Please **never** run this command on the login node
- Use and adapt `start_gym.sbatch` to train on the cluster
- Set `--nv` in your `.sbatch` file if you want to use a GPU unit for training.
- Add argument `--no_display` to hide the window (cluster)
- Start with the provided default parameters
- Training produces two plots
  - Loss curve
  - Episode rewards

e) Evaluation

Evaluate the trained deep Q-learning agent:

- Use script `evaluate_racing.py` to evaluate it.
- Use argument `--no_display` when you want to run this without a visible display.
- Please **never** run this command on the login node.
- Read descriptions of other arguments.
- Script will output preliminary leaderboard score
- Visual evaluation on local machine should be performed

Important: Make sure you have a working baseline implementation (i.e. agent is able to take some corners) before moving on to work on the next part of the exercise.

1.2
Further Investigations and Extensions

Extensions

- a) Discount factor
- b) Action repeat parameter
- c) Action space
- d) Double Q-Learning

d) Double Q-learning

Implement double Q-learning:

- Implement it in `learning.py`.
- Use argument `--use_doubleqlearning` to enable double Q learning.

Important: Include your double Q-learning implementation in your code submission

Competition

Competition

Submit your evaluation scores to the leaderboard:

- Make sure not to overfit on the provided evaluation tracks
- Final evaluation will be performed by us on a secret set of tracks
- Winners will present their approach in the last lecture

Advice

Advice

Read the DQN papers on playing Atari games

- In particular, the Nature paper by Mnih et al. (2015)
- Check their pseudo-code

Start early

- Training a reinforcement learning takes time (several hours)
- You will need to train and evaluate several agents for this exercise
- Don’t start too late or you will run out of time

Questions?